LAPORAN

WORKSHOP KECERDASAN BUATAN



RZAINURTANTO

5324600024

PROGRAM STUDI SARJANA TERAPAN TEKNOLOGI REKAYASA MULTIMEDIA

JURUSAN TEKNOLOGI MULTIMEDIA KREATIF

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

A. MATERI

- Machine Learning

B. ALAT DAN BAHAN

- Laptop
- VsCode
- GitHub
- Word

C. PERCOBAAN

1. Example 3.1

```
Bab3 > example3.1.py > ...
1   #Example 3.1 Python SVM Classifications
2   from sklearn import svm
3   X = [[170, 70, 10], [180, 80,12], [170, 65, 8],[160, 55, 7]]
4   #Height[cm], Weight[kg], Shoesize[UK]
5   y = [0, 0, 1, 1] #Gender, 0: Male, 1: Female
6   clf = svm.SVC()
7   clf.fit(X, y)
8   #Predict
9   p = clf.predict([[160, 60, 7]])
10  print(p)
```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kece
xe" "d:/AI/Kecerdasan Buatan/Bab3/example3.1.py"
[1]
```

2. Example 3.2

```
Bab3 > Code > example3.2.py > ...
1   #Example 3.2 Python SVM Iris Classifications
2   from sklearn import svm, datasets
3   iris = datasets.load_iris()
4   # Take the first two features: Sepal length and Sepal width
5   X = iris.data[:, :2]
6   y = iris.target #0: Setosa, 1: Versicolour, 2:Virginica
7   print(y)
8   clf = svm.SVC()
9   clf.fit(X, y)
10  #Predict the flower for a given Sepal length and width
11  p = clf.predict([[5.4, 3.2]])
12  print(p)
```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan Buatan\.venv\Scripts
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.2.py"
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]
[0]
```

3. Example 3.3

```
Bab3 > Code > example3.3.py > ...
1   #Example 3.3 Python SVM Iris CSV Classifications
2   from sklearn import svm, datasets
3   import pandas as pd
4   df = pd.read_csv('iris.csv')
5   X = df.values[:,:2]
6   s = df['species']
7   d = dict([(y,x) for x,y in enumerate(sorted(set(s)))])
8   y = [d[x] for x in s]
9   clf = svm.SVC()
10  clf.fit(X, y)
11  #Predict the flower for a given Sepal length and width
12  p = clf.predict([[5.4, 3.2]])
13  print(p)
```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerda
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.3.py
[0]
```

4. Example 3.4

Bab3 > Code > example3.4.py > ...

```python
# Example 3.4 Python SVM Iris URL Classifications
from sklearn import svm
import pandas as pd
from matplotlib import pyplot
from pandas.plotting import scatter_matrix


df = pd.read_csv(
    "https://gist.githubusercontent.com/curran/"
    "a08a1080b88344b0c8a7/raw/0e7a9b0a5d22642a06d3d5b9bcbad9890c8ee534/"
    "iris.csv"
)
print(df.shape)
print(df.head(10))
print(df.tail(10))
print(df.describe())

print(df.isna().sum().sum())
df = df.dropna()
print(df.groupby("species").size())

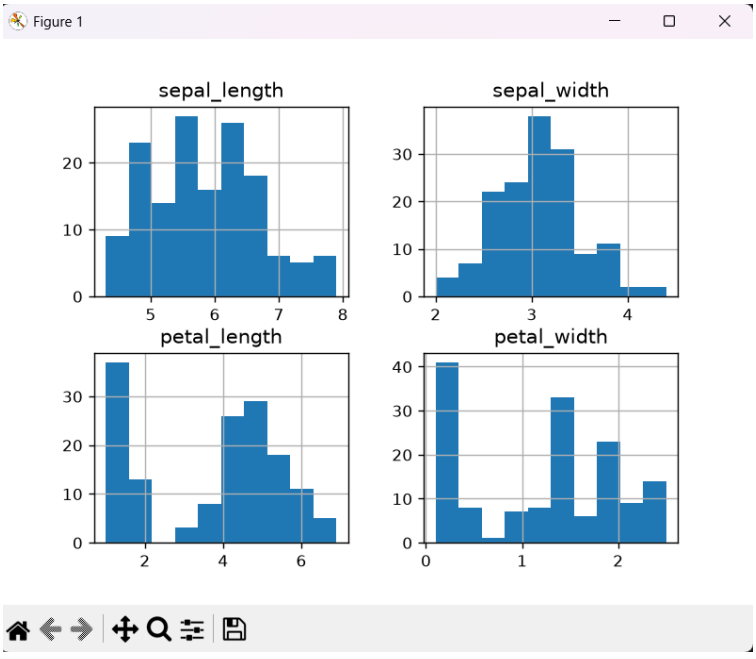
df.hist()
pyplot.show()

scatter_matrix(df)
pyplot.show()

X = df.values[:, :2]
species = df["species"]
labels = dict((name, index) for index, name in enumerate(sorted(set(species))))
y = [labels[name] for name in species]

clf = svm.SVC()
clf.fit(X, y)
p = clf.predict([[5.4, 3.2]])
print(p)
```

Output:



5. Example 3.5

```
Bab3 > Code > example3.5.py > ...
1   # Example 3.5 Breast Cancer Data
2   from sklearn.datasets import load_breast_cancer
3
4
5   cancer = load_breast_cancer()
6   print(cancer["feature_names"])
7   print(cancer["data"])
8   print(cancer.target_names)
9
```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan Buatan\.venv\Scripts\python.e
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.5.py"
['mean radius' 'mean texture' 'mean perimeter' 'mean area'
 'mean smoothness' 'mean compactness' 'mean concavity'
 'mean concave points' 'mean symmetry' 'mean fractal dimension'
 'radius error' 'texture error' 'perimeter error' 'area error'
 'smoothness error' 'compactness error' 'concavity error'
 'concave points error' 'symmetry error' 'fractal dimension error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst smoothness' 'worst compactness' 'worst concavity'
 'worst concave points' 'worst symmetry' 'worst fractal dimension']
[[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01 8.758e-02]
 ...
 [1.660e+01 2.808e+01 1.083e+02 ... 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 ... 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 ... 0.000e+00 2.871e-01 7.039e-02]]
['malignant' 'benign']
(.venv) PS D:\AI\Kecerdasan Buatan>
```

6. Example 3.6

Bab3 > Code > example3.6.py > ...

```python
# Example 3.6 Python SVM Breast Cancer Classifications
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC


cancer = load_breast_cancer()
X = cancer.data
y = cancer.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=20
)
clf = SVC()
clf.fit(X_train, y_train)
y_predict = clf.predict(X_test)

cm = np.array(confusion_matrix(y_test, y_predict, labels=[1, 0]))
confusion = pd.DataFrame(
    cm,
    index=["is_cancer", "is_healthy"],
    columns=["predicted_cancer", "predicted_healthy"],
)
print(confusion)
print(classification_report(y_test, y_predict))

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.6.py"
           1       0.89      1.00      0.94        66

    accuracy                           0.93       114
   macro avg       0.95      0.92      0.93       114
weighted avg       0.94      0.93      0.93       114
```

7. Example 3.7

Bab3 > Code > example3.7.py > ...

```python
# Example 3.7 Naive Bayes Iris
import joblib
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB


X, y = load_iris(return_X_y=True)
clf = GaussianNB()
clf.fit(X, y)
joblib.dump(clf, "model.pkl")
clf2 = joblib.load("model.pkl")
p = clf2.predict([[5.0, 3.4, 1.5, 0.4]])
print(p)

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan>
...
                                    & "d:\AI\Kecerdasan Buatan\.venv\Scripts\python.e
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.7.py"
[0]
```

8. Example 3.8

Bab3 > Code > example3.8.py > ...

```python
# Example 3.8 LinearDiscriminantAnalysis Classification
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis


X, y = make_classification(
    n_samples=2000,
    n_features=6,
    n_informative=3,
    n_redundant=0,
    random_state=0,
    shuffle=False,
)
print(X)
clf = LinearDiscriminantAnalysis()
clf.fit(X, y)
print(clf.predict([[0, 0, 0, 0, 0, 0]]))

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan Buatan\.venv\Scripts\python.e
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.8.py"
[[-0.59734739  0.48399984 -2.34065228 -0.60004987  0.44629917  0.53769498]
 [-1.10731738  0.66539105 -2.3302963  -0.81848651  0.19724341  2.00734314]
 [ 0.48993689  1.30537777 -1.01582594 -1.08660921  1.34541311 -1.81702209]
 ...
 [ 0.55858636  1.20745514  1.55956083 -0.50435786 -0.32014316 -0.17555345]
 [ 0.00642618  0.93435542  2.19154679  1.1466348  -1.92120109 -0.2301177 ]
 [-0.2778356   0.81649127  2.22615366  1.11839564  0.03791157 -1.08083163]]
[0]
```

9. Example 3.9

```
Bab3 > Code > example3.9.py > ...
1   # Example 3.9 Principal Component Analysis
2   from sklearn.datasets import make_classification
3   from sklearn.decomposition import PCA
4
5
6   X, y = make_classification(
7       n_samples=1000,
8       n_features=4,
9       n_informative=2,
10      n_redundant=0,
11      random_state=0,
12      shuffle=False,
13  )
14  print(X)
15  clf = PCA()
16  clf.fit(X, y)
17  print(clf.explained_variance_ratio_)
18  print(clf.singular_values_)
19
```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerda
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.9.py
[[-1.66853167 -1.29901346  0.2746472  -0.60362044]
 [-2.9728827  -1.08878294  0.70885958  0.42281857]
 [-0.59614125 -1.37007001 -3.11685659  0.64445203]
 ...
 [ 0.91711204  1.10596645  0.86766522 -2.25625012]
 [ 0.10027664  1.45875846 -0.44360274 -0.67002328]
 [ 1.0415229  -0.01987143  0.15216419 -1.9405334 ]]
[0.35054565 0.26955436 0.1920725  0.1878275 ]
[42.31488145 37.10598956 31.32228453 30.97422379]
```

10. Example 3.10

Bab3 > Code > example3.10.py > ...

```python
# Example 3.10 Principal Component Analysis Iris
import matplotlib.pyplot as plt
from sklearn import decomposition, datasets


iris = datasets.load_iris()
X = iris.data
y = iris.target

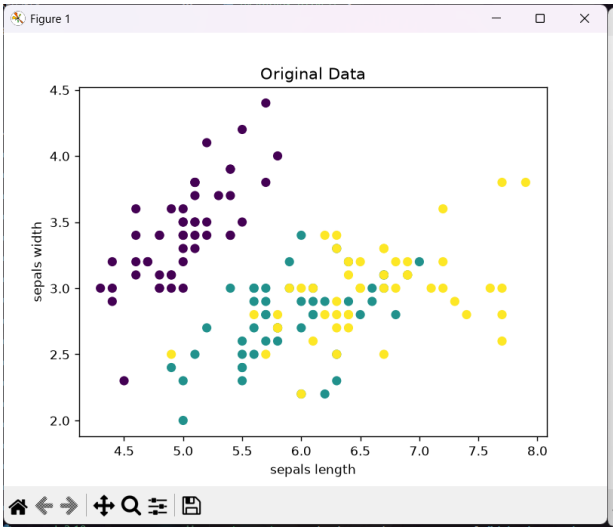
f = plt.figure(1)
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel("sepals length")
plt.ylabel("sepals width")
plt.title("Original Data")
f.show()

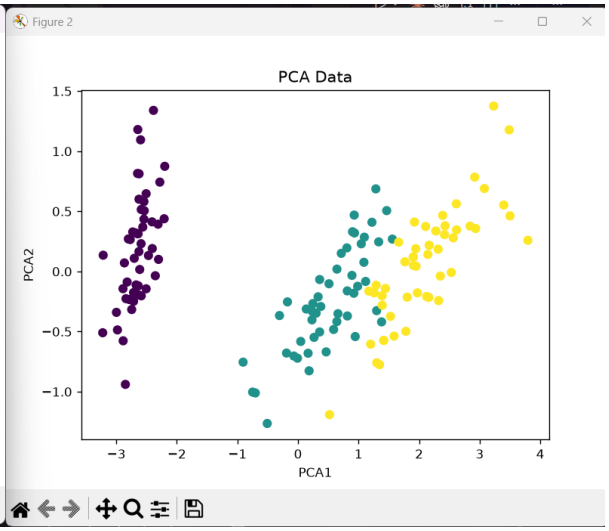
pca = decomposition.PCA(n_components=3)
pca.fit(X)
X1 = pca.transform(X)

g = plt.figure(2)
plt.scatter(X1[:, 0], X1[:, 1], c=y)
plt.xlabel("PCA1")
plt.ylabel("PCA2")
plt.title("PCA Data")
g.show()

```

Output:





11. Example 3.11

```python
# Example 3.11 Decision Tree Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier


X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=0
)
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
N = y_test.shape[0]
C = (y_test == y_pred).sum()
print("Total points: %d Correctly labeled points : %d" % (N, C))

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan Buatan\.venv\Scripts\python.exe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.11.py"
Total points: 75 Correctly labeled points : 72
```

12. Example 3.12

```python
# Example 3.12 Random Forest
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier


X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=0)
clf = RandomForestClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Total points: %d Correctly labeled points : %d" % (y_test.shape[0], (y_test == y_pred).sum()))

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.12.py"
Total points: 75 Correctly labeled points : 71
```

13. Example 3.13

```python
# Example 3.13 K-Nearest Neighbors Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier


X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=0)
clf = KNeighborsClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Total points: %d Correctly labeled points : %d" % (y_test.shape[0], (y_test == y_pred).sum()))

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerd
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.13.
Total points: 75 Correctly labeled points : 72
```

14. Example 3.14

```python
# Example 3.14 Comparison of Different Classification
from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier


names = [
    "SVM",
    "Naive Bayes",
    "LDA",
    "QDA",
    "Decision Tree",
    "Random Forest",
    "Nearest Neighbors",
    "Neural Networks",
]
classifiers = [
    SVC(),
    GaussianNB(),
    LinearDiscriminantAnalysis(),
    QuadraticDiscriminantAnalysis(),
    DecisionTreeClassifier(),
    RandomForestClassifier(),
    KNeighborsClassifier(),
    MLPClassifier(alpha=1, max_iter=1000),
]

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=0)

for name, clf in zip(names, classifiers):
    clf.fit(X_train, y_train)
    score = clf.score(X_test, y_test)
    print(name + ": " + str(score))

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> &
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code
SVM: 0.9466666666666667
Naive Bayes: 0.9466666666666667
LDA: 0.96
QDA: 0.96
Decision Tree: 0.96
Random Forest: 0.9466666666666667
Nearest Neighbors: 0.96
Neural Networks: 0.9866666666666667
(.venv) PS D:\AI\Kecerdasan Buatan>
```

15. Example 3.15

Bab3 > Code > example3.15.py > ...

```python
# Example 3.15 Linear Regression
import matplotlib.pyplot as plt
from scipy import stats

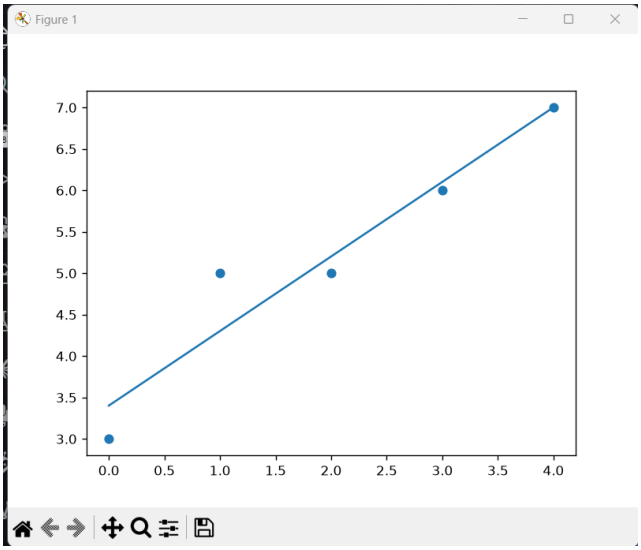

x = [0, 1, 2, 3, 4]
y = [3, 5, 5, 6, 7]

slope, intercept, r, p, std_err = stats.linregress(x, y)
print("slope: ", slope)
print("intercept: ", intercept)


def myfunc(x):
    return slope * x + intercept


mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```

Output:



16. Example 3.15a

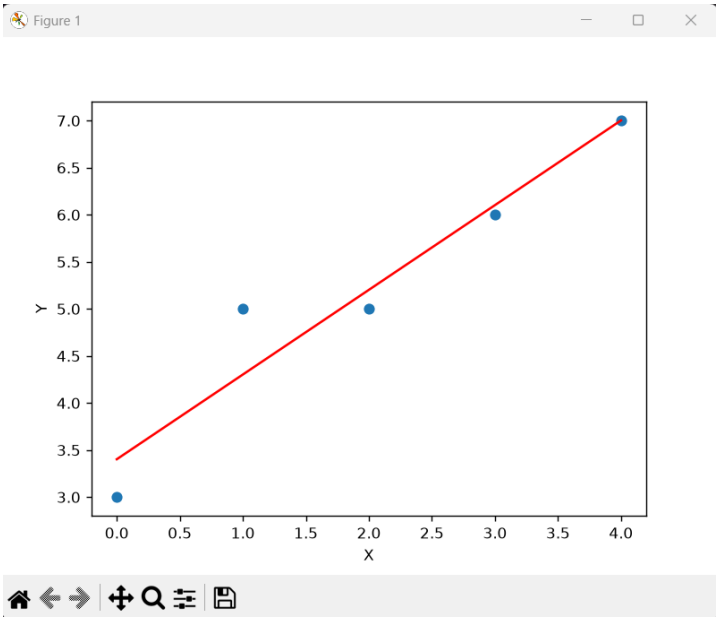
Bab3 > Code > example3.15a.py > ...

```python
# Example 3.15a Linear Regression
import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm


x = [0, 1, 2, 3, 4]
y = [3, 5, 5, 6, 7]
x1 = sm.add_constant(x)
model = sm.OLS(y, x1)
results = model.fit()
print(results.params)
print(results.summary())
y_pred = results.predict(x1)
plt.scatter(x, y)
plt.xlabel("X")
plt.ylabel("Y")
plt.plot(x, y_pred, "r")
plt.show()

```

Output:



17. Example 3.16

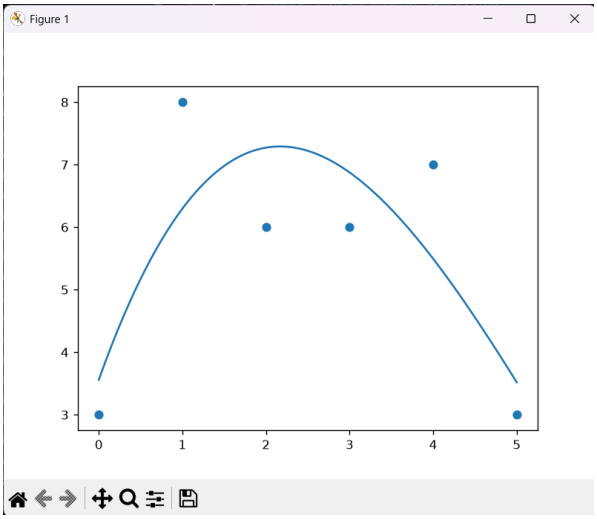
Bab3 > Code > example3.16.py > ...

```python
# Example 3.16 Polynomial Regression
import matplotlib.pyplot as plt
import numpy as np


x = [0, 1, 2, 3, 4, 5]
y = [3, 8, 6, 6, 7, 3]
mymodel = np.poly1d(np.polyfit(x, y, 3))
print(mymodel)
myline = np.linspace(0, 5, 100)
plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()

```

Output:



18. Example 3.17

Bab3 > Code > example3.17.py > ...

```python
# Example 3.17 Least Squares Fitting
import numpy as np
import scipy.optimize as optimization


x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([100, 90, 60, 30, 10, 1])


def func(x, a, b, c):
    return a * np.exp(-b * x) + c


popt, pcov = optimization.curve_fit(func, x, y)
print("Best fit a b c: ", popt)
print("Best fit covariance: ", pcov)

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan Buatan\.venv\Scr
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.17.py"
ered in exp
  return a * np.exp(-b * x) + c
Best fit a b c:  [ 4.09564521e+02  6.21465806e-02 -3.04106842e+02]
Best fit covariance:  [[ 4.92893733e+05 -8.73855445e+01 -4.95831373e+05]
 [-8.73855445e+01  1.55361514e-02  8.79419359e+01]
 [-4.95831373e+05  8.79419359e+01  4.98827106e+05]]
(.venv) PS D:\AI\Kecerdasan Buatan>
```

19. Example 3.18

Bab3 > Code > example3.18.py > ...

```python
# Example 3.18 Multiple Linear Regression
from sklearn import linear_model
import numpy as np


x = np.array([[0, 3, 5], [1, 4, 6], [2, 5, 7], [3, 6, 8], [4, 7, 9]])
y = np.array([3, 5, 5, 6, 7])
reg = linear_model.LinearRegression()
reg.fit(x, y)
print('Coefficients: \n', reg.coef_)
print('Intercept: \n', reg.intercept_)
pred = reg.predict([[5, 8, 10]])
print('Predition: \n', pred)

```

Output:

```
(.venv) PS D:\AI\Ke
xe" "d:/AI/Kecerdas
Coefficients:
 [0.3 0.3 0.3]
Intercept:
 0.9999999999999991
Predition:
 [7.9]
```

20. Example 3.19

Bab3 > Code > example3.19.py > ...

```python
# Example 3.19 Logistic Regression
import numpy as np
from sklearn.linear_model import LogisticRegression


X = np.array([[0], [1], [2], [3], [4], [5]])
y = np.array([1, 2, 3, 30, 32, 31])
clf = LogisticRegression(random_state=0).fit(X, y)
print(clf.predict([[6]]))
print(clf.predict_proba([[6]]))
print(clf.score(X, y))

```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan Buatan\.venv\S
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.19.py"
[31]
[[0.00111396 0.00810048 0.03285548 0.09767681 0.61033535 0.24991792]]
0.6666666666666666
(.venv) PS D:\AI\Kecerdasan Buatan>
```

21. Example 3.20

```
Bab3 > Code > example3.20.py > ...
1   # Example 3.20 K-means Clustering
2   from sklearn.cluster import KMeans
3   import numpy as np
4
5
6   X = np.array([[1, 2, 3], [1, 4, 2], [1, 0, 3], [10, 2, 4], [9, 4, 3], [11, 0, 2]])
7   kmeans = KMeans(n_clusters=2, random_state=0).fit(X)
8   print(kmeans.labels_)
9   print(kmeans.cluster_centers_)
10  print(kmeans.predict([[12, 3, 1]]))
11
```

Output:

```
0.6666666666666666
(.venv) PS D:\AI\Kecerdasan Buatan> & "d
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/e
[1 1 1 0 0 0]
[[10.          2.          3.        ]
 [ 1.          2.          2.66666667]]
[0]
(.venv) PS D:\AI\Kecerdasan Buatan>
```

22. Example 3.21

```
Bab3 > Code > example3.21.py > ...
1   # Example 3.21 Semi-supervised Learning
2   import numpy as np
3   from sklearn.semi_supervised import LabelSpreading
4
5
6   X = np.array([[0, 1], [1, 1], [2, 0], [3, 1], [10, 5], [11, 6], [12, 4], [13, 5]])
7   y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
8   labels = np.full(8, -1.0)
9   labels[0] = 0
10  labels[-1] = 1
11  print(labels)
12
13  label_spread = LabelSpreading(kernel='knn', alpha=0.8)
14  label_spread.fit(X, labels)
15  output_labels = label_spread.transduction_
16  print(output_labels)
17
```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> &
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code
[ 0. -1. -1. -1. -1. -1. -1.  1.]
[0. 0. 0. 0. 1. 1. 1. 1.]
(.venv) PS D:\AI\Kecerdasan Buatan>
```

23. Example 3.22

example3.10.py U example3.22.py U

Bab3 > Code > example3.22.py > ...

```python
# Example 3.22 - Q_test.py
# Modified from:
# https://amunategui.github.io/reinforcement-learning/index.html
# http://firsttimeprogrammer.blogspot.com/2016/09/getting-ai-smarter-with-q-learning.html
# http://mnemstudio.org/path-finding-q-learning-tutorial.htm
import numpy as np
import pylab as plt
from Q_Utils import *

# Setting up Parameters ========================================
# Create a routing list and set the goal
points_list = [(0, 1), (1, 2), (1, 3), (2, 4), (3, 5), (3, 6)]
goal = 6
# Show the routing graph
showgraph(points_list)
# The number of points of the R matrix
MATRIX_SIZE = 7
# create matrix R
R = createRmat(MATRIX_SIZE, points_list, goal)
# Create matrix Q
Q = np.matrix(np.zeros([MATRIX_SIZE, MATRIX_SIZE]))
# Learning parameter
gamma = 0.8

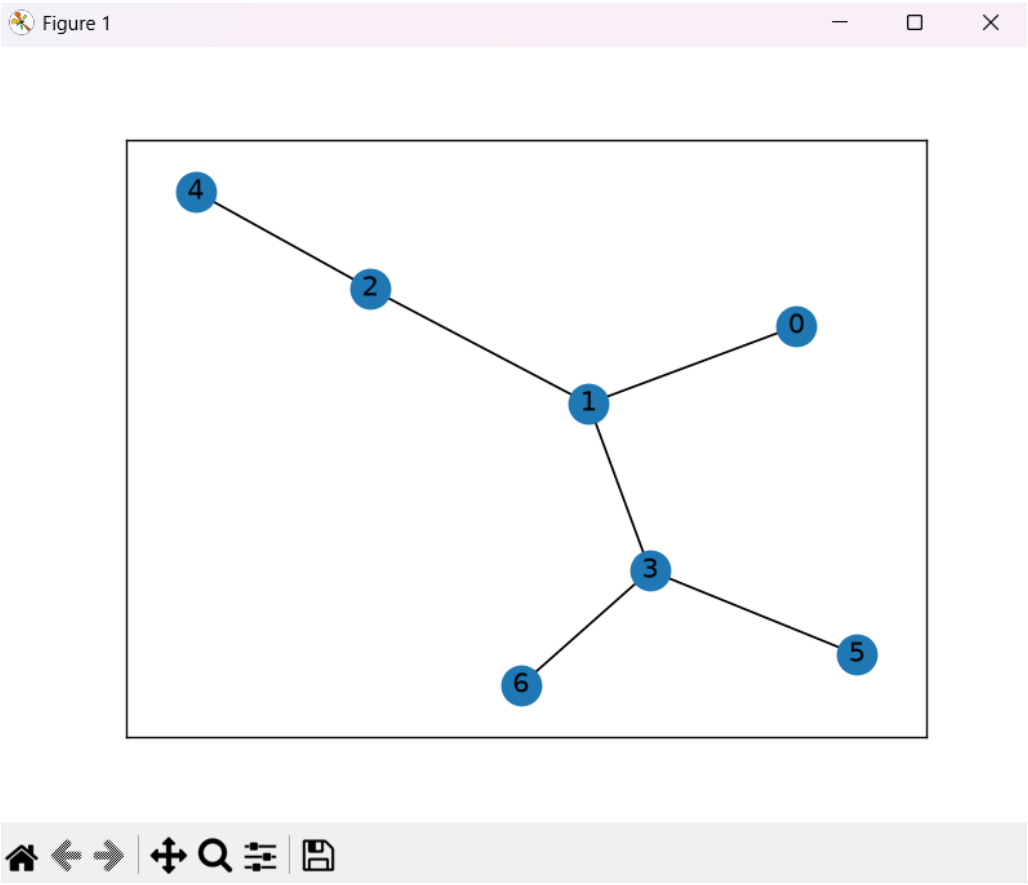
# Training =====================================================
scores = []
for i in range(700):
    # Select a random current_state (starting point)
    current_state = np.random.randint(0, int(Q.shape[0]))
    # Work out all the available next step actions
    available_act = available_actions(R, current_state)
    # Choose a random next step action
    action = sample_next_action(available_act)
    # Update the Q matrix
    score = update(R, Q, current_state, action, gamma)
    scores.append(score)
    print('Score:', str(score))

print("Trained Q matrix:")
print(Q / np.max(Q) * 100)

# Testing ======================================================
current_state = 0
steps = [current_state]
while current_state != goal:
    next_step_index = np.where(Q[current_state,] == np.max(Q[current_state,]))[1]
    if next_step_index.shape[0] > 1:
        next_step_index = int(np.random.choice(next_step_index, size=1))
    else:
        next_step_index = int(next_step_index)
    steps.append(next_step_index)
    current_state = next_step_index

# Display Results ==============================================
print("Most efficient path:")
print(steps)
plt.plot(scores)
plt.show()
```

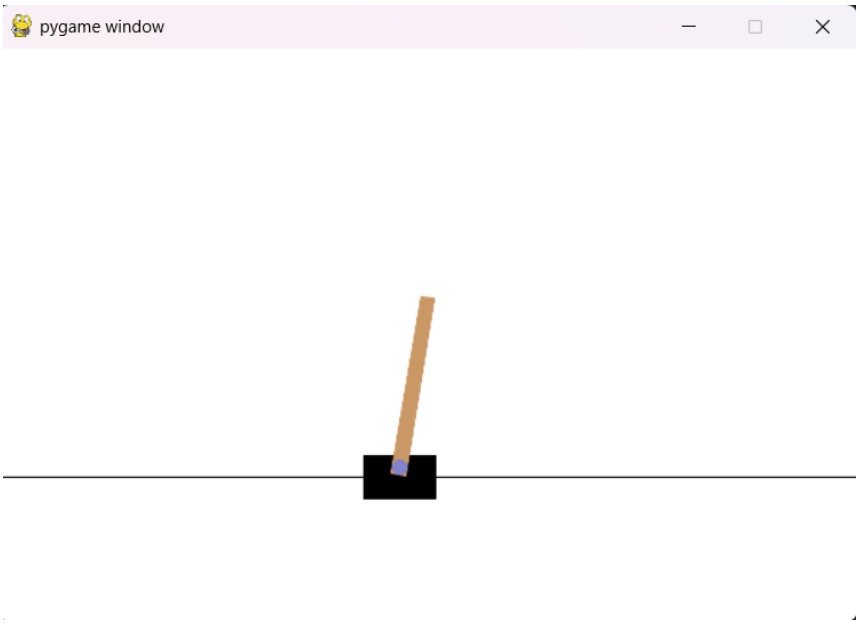
Output:



24. Example 3.23

```
Bab3 > Code > example3.23.py > ...
1   # Example 3.23 OpenAI Gym CartPole
2   # https://gym.openai.com/docs/
3   # https://gym.openai.com/envs/CartPole-v0/
4   import gym
5
6
7   env = gym.make('CartPole-v0')
8   for i_episode in range(20):
9       observation = env.reset()
10      for t in range(100):
11          env.render()
12          print(observation)
13          action = env.action_space.sample()
14          observation, reward, done, info = env.step(action)
15          if done:
16              print("Episode finished after {} timesteps".format(t + 1))
17              break
18
19  env.close()
20
```

Output:



25. Example 3.24

```
Bab3 > Code > example3.24.py > ...
1   # Example 3.24 Ensemble1.py
2   from sklearn import datasets
3   from sklearn.ensemble import RandomForestClassifier, VotingClassifier
4   from sklearn.linear_model import LogisticRegression
5   from sklearn.model_selection import cross_val_score
6   from sklearn.naive_bayes import GaussianNB
7   from sklearn.svm import SVC
8
9
10  iris = datasets.load_iris()
11  X, y = iris.data[:, 1:3], iris.target
12  clf1 = LogisticRegression(random_state=1)
13  clf2 = RandomForestClassifier(n_estimators=50, random_state=1)
14  clf3 = GaussianNB()
15  clf4 = SVC()
16  eclf = VotingClassifier(
17      estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3), ('svc', clf4)],
18      voting='hard',
19  )
20
21  for clf, label in zip(
22      [clf1, clf2, clf3, clf4, eclf],
23      ['Logistic Regression', 'Random Forest', 'naive Bayes', 'SVM', 'Ensemble'],
24  ):
25      scores = cross_val_score(clf, X, y, scoring='accuracy', cv=5)
26      print("Accuracy: %0.2f (+/- %0.2f) [%s]" % (scores.mean(), scores.std(), label))
27
```

Output:

```
(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\K
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/example
Accuracy: 0.95 (+/- 0.04) [Logistic Regression
Accuracy: 0.94 (+/- 0.04) [Random Forest]
Accuracy: 0.91 (+/- 0.04) [naive Bayes]
Accuracy: 0.95 (+/- 0.04) [SVM]
Accuracy: 0.95 (+/- 0.04) [Ensemble]
(.venv) PS D:\AI\Kecerdasan Buatan>
```

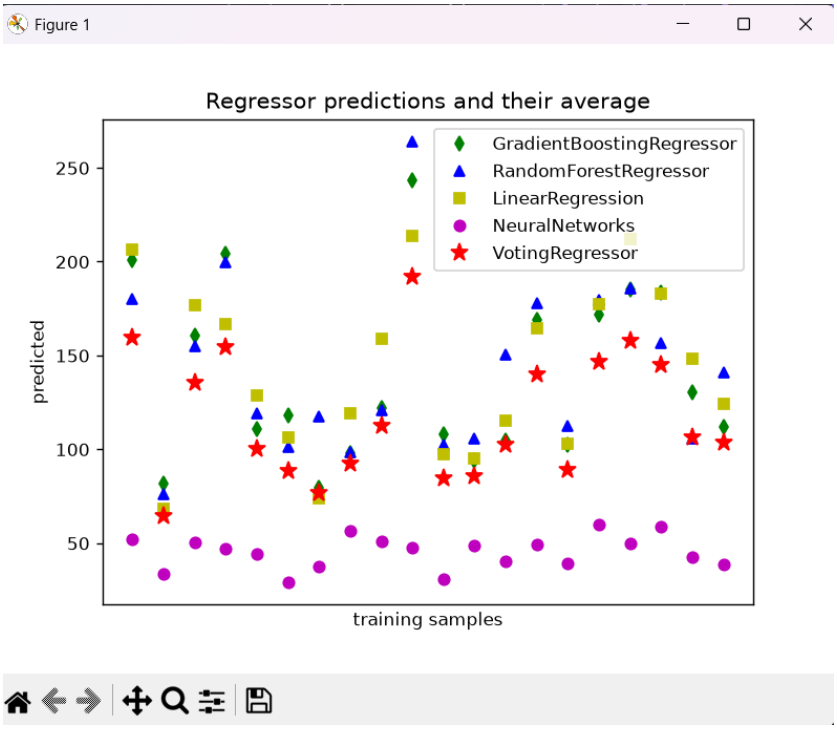
26. Example 3.25

```python
# Example 3.25 Ensemble2.py
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor, VotingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.neural_network import MLPRegressor


X, y = load_diabetes(return_X_y=True)
reg1 = GradientBoostingRegressor(random_state=1)
reg2 = RandomForestRegressor(random_state=1)
reg3 = LinearRegression()
reg4 = MLPRegressor()
reg1.fit(X, y)
reg2.fit(X, y)
reg3.fit(X, y)
reg4.fit(X, y)
ereg = VotingRegressor(estimators=[('gb', reg1), ('rf', reg2), ('lr', reg3), ('NN', reg4)])
print(ereg.fit(X, y))

xt = X[:20]
pred1 = reg1.predict(xt)
pred2 = reg2.predict(xt)
pred3 = reg3.predict(xt)
pred4 = reg4.predict(xt)
pred5 = ereg.predict(xt)
plt.figure()
plt.plot(pred1, 'gd', label='GradientBoostingRegressor')
plt.plot(pred2, 'b^', label='RandomForestRegressor')
plt.plot(pred3, 'ys', label='LinearRegression')
plt.plot(pred4, 'mo', label='NeuralNetworks')
plt.plot(pred5, 'r*', ms=10, label='VotingRegressor')
plt.tick_params(axis='x', which='both', bottom=False, top=False, labelbottom=False)
plt.ylabel('predicted')
plt.xlabel('training samples')
plt.legend(loc='best')
plt.title('Regressor predictions and their average')
plt.show()
```

Output:



27. Example 3.26

```
Bab3 > Code > example3.26.py > ...
1   # Example 3.26 Auto_ml_test.py
2   from auto_ml import Predictor
3   from auto_ml.utils import get_boston_dataset
4
5
6   df_train, df_test = get_boston_dataset()
7   column_descriptions = {
8       'MEDV': 'output',
9       'CHAS': 'categorical',
10  }
11  ml_predictor = Predictor(type_of_estimator='regressor', column_descriptions=column_descriptions)
12  ml_predictor.train(df_train)
13  ml_predictor.score(df_test, df_test.MEDV)
14
```

Output:

```
>> & "d:/AI/Kecerdasan Buatar
R2 score: 0.8059
```

28. Example 3.27

```
(.venv311) PS D:\AI\Kecerdasan Buatan> & C
on\Python311\python.exe "d:/AI/Kecerdasan
lr      0.9599  0.9408  0.9458     0.610
knn     0.9508  0.9284  0.9361     0.034
nb      0.9507  0.9270  0.9332     0.016
rf      0.9507  0.9270  0.9332     0.057
gbc     0.9507  0.9270  0.9332     0.079
et      0.9417  0.9133  0.9183     0.046
dt      0.9410  0.9119  0.9167     0.014
ada     0.9410  0.9119  0.9167     0.030
ridge   0.8397  0.7688  0.7866     0.009
svm     0.7087  0.6343  0.6962     0.016
dummy   0.1277  0.0000  0.0000     0.010
```

29. Example 3.28a

```
(.venv311) PS D:\AI\Kecerdasan Buatan> !pip install lazypredit
```

Output:

30. Example 3.28b

```
Bab3 > Code > example3.28b.py
1   # Example 3.28b The LazyPredict.ipynb Program (Part 2)
2   import lazypredict
3   from lazypredict.Supervised import LazyClassifier
4   from sklearn.datasets import load_iris
5   from sklearn.model_selection import train_test_split
6
7
8   X, y = load_iris(return_X_y=True)
9   X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.25, random_state=1)
10  clf = LazyClassifier()
11  models, predictions = clf.fit(X_train, X_test, y_train, y_test)
12  print(models)
13
```

Output:

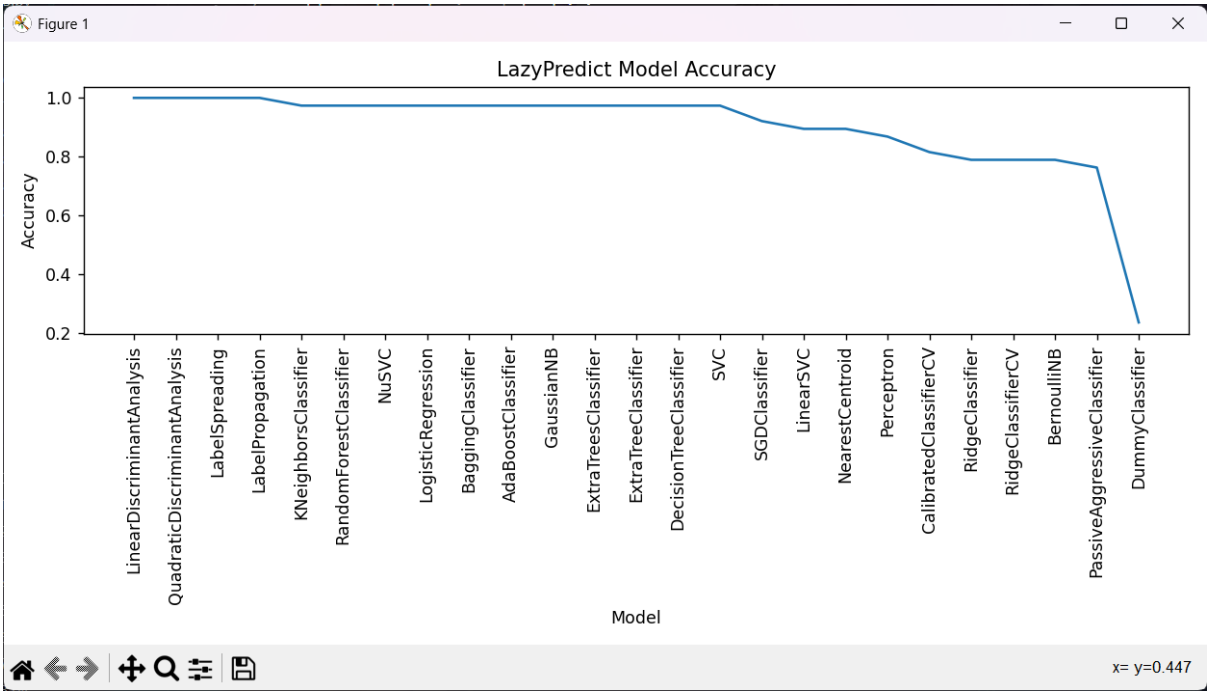
```
(.venv311) PS D:\AI\Kecerdasan Buatan> & C:\Users\welcome\AppData\Local\Programs\Python\Python311\python.exe "d:/AI/Kecerdasan Buatan/Bab3/Code/example3.28b.py"
LinearSVC                      0.894737           0.916667  ...  0.894737    0.010404
NearestCentroid                0.894737           0.900463  ...  0.894737    0.129952
Perceptron                     0.868421           0.895833  ...  0.868421    0.009634
CalibratedClassifierCV         0.815789           0.854167  ...  0.815789    0.038837
RidgeClassifier                0.789474           0.833333  ...  0.789474    0.009658
RidgeClassifierCV              0.789474           0.833333  ...  0.789474    0.012863
BernoulliNB                    0.789474           0.817130  ...  0.789474    0.010000
PassiveAggressiveClassifier    0.763158           0.812500  ...  0.763158    0.011128
DummyClassifier                0.236842           0.333333  ...  0.236842    0.010008

[25 rows x 7 columns]
```

31. Example 3.28c

```
Bab3 > Code > example3.28c.py
1  # Example 3.28c The LazyPredict.ipynb Program
2  import matplotlib.pyplot as plt
3
4  plt.figure(figsize=(10, 5))
5  plt.plot(models.index, models['Accuracy'])
6
```

Output:



32. Example 3.28d

Bab3 > Code > example3.28d.py

```python
# Example 3.28d The LazyPredict.ipynb Program (Part 4)
import lazypredict
from lazypredict.Supervised import LazyRegressor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

X, y = fetch_california_housing(return_X_y=True, as_frame=True)
X = X.head(5000).copy()
y = y.head(5000).copy()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2, random_state=1)
reg = LazyRegressor(verbose=0, ignore_warnings=True)
models, predictions = reg.fit(X_train, X_test, y_train, y_test)
print(models.head())

```

Output:

```
                           Adjusted R-Squared  R-Squared      RMSE  Time Taken
Model
RandomForestRegressor                0.858059   0.859196  0.413080    4.539807
ExtraTreesRegressor                  0.855706   0.856861  0.416491    2.141619
HistGradientBoostingRegressor        0.855622   0.856778  0.416611    1.820510
BaggingRegressor                     0.849135   0.850343  0.425869    0.465148
GradientBoostingRegressor            0.838594   0.839887  0.440495    2.489753
(.venv311) PS D:\AI\Kecerdasan Buatan>
```

33. Example 3.28e

Bab3 > Code > example3.28e.py

```python
# Example 3.28e The LazyPredict.ipynb Program (Part 5)
import matplotlib.pyplot as plt
from lazypredict.Supervised import LazyRegressor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

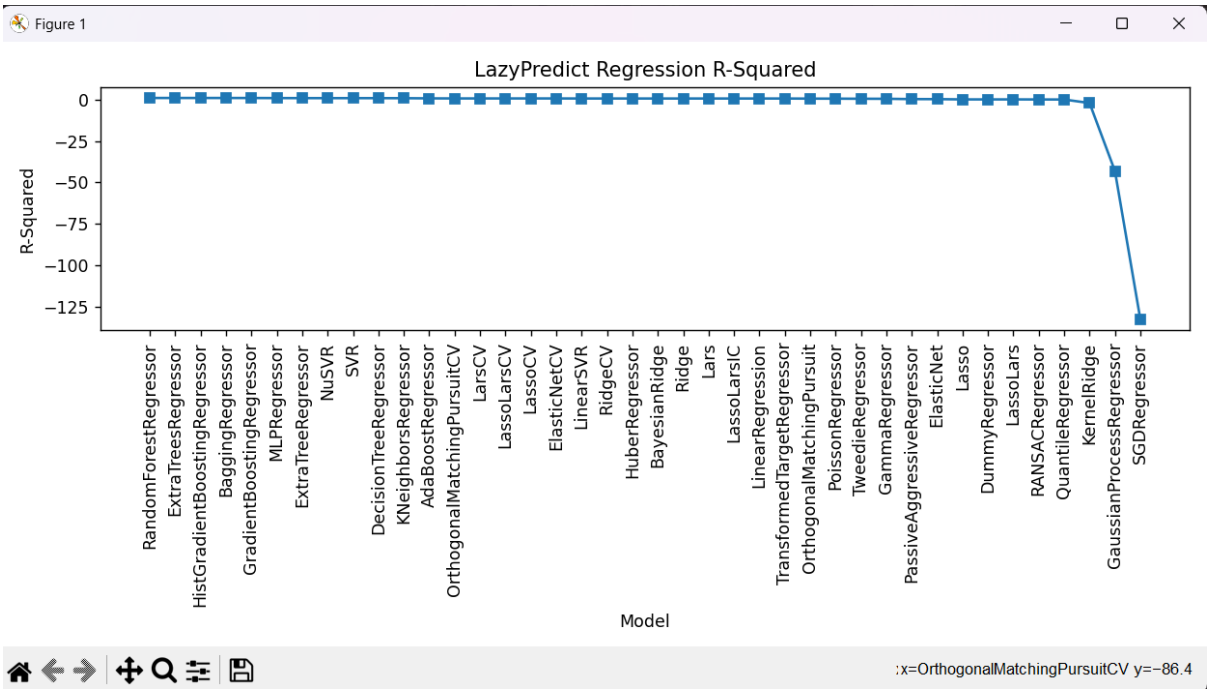
X, y = fetch_california_housing(return_X_y=True, as_frame=True)
X = X.head(5000).copy()
y = y.head(5000).copy()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2, random_state=1)
reg = LazyRegressor(verbose=0, ignore_warnings=True)
models, predictions = reg.fit(X_train, X_test, y_train, y_test)

plt.figure(figsize=(10, 5))
plt.plot(models.index, models['R-Squared'], '-s')
plt.xticks(rotation=90)
plt.ylabel('R-Squared')
plt.xlabel('Model')
plt.title('LazyPredict Regression R-Squared')
plt.tight_layout()
plt.show()

```

Ouput:



D. LATIHAN

1. Exercise 3.1

```
Bab3 > Code > exercise3.1.py > ...
# Exercise 3.1 Python SVM Classification
from sklearn import svm

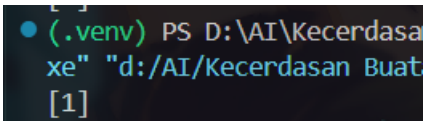
X = [
    [170, 70, 10],
    [180, 80, 12],
    [170, 65, 8],
    [160, 55, 7],
    [175, 72, 9],
    [158, 52, 6]
]
# [Height cm, Weight kg, Shoesize UK]

y = [0, 0, 1, 1, 0, 1]
# Gender, 0: Male, 1: Female

clf = svm.SVC()
clf.fit(X, y)

# Predict
p = clf.predict([[160, 60, 7]])
print(p)
```

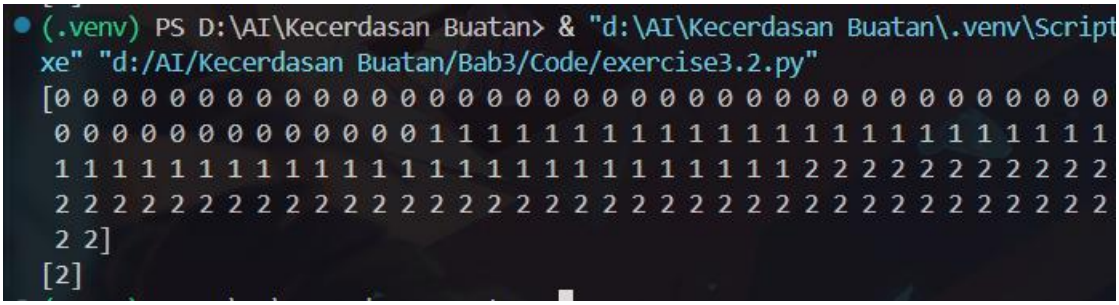
Output:



2. Exercise 3.2



Output:


(.venv) PS D:\AI\Kecerdasan Buatan> & "d:\AI\Kecerdasan Buatan\.venv\Script
xe" "d:/AI/Kecerdasan Buatan/Bab3/Code/exercise3.2.py"
[0 0 0

3. Exercise 3.3

Bab3 > Code > exercise3.3.py

```python
# Exercise 3.3 Python SVM Iris URL Scatter Plot
from sklearn import svm
import pandas as pd
from matplotlib import pyplot


df = pd.read_csv(
    'https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/raw/0e7a9b0a5d22642a06d3d5b9bcbad9890c
)

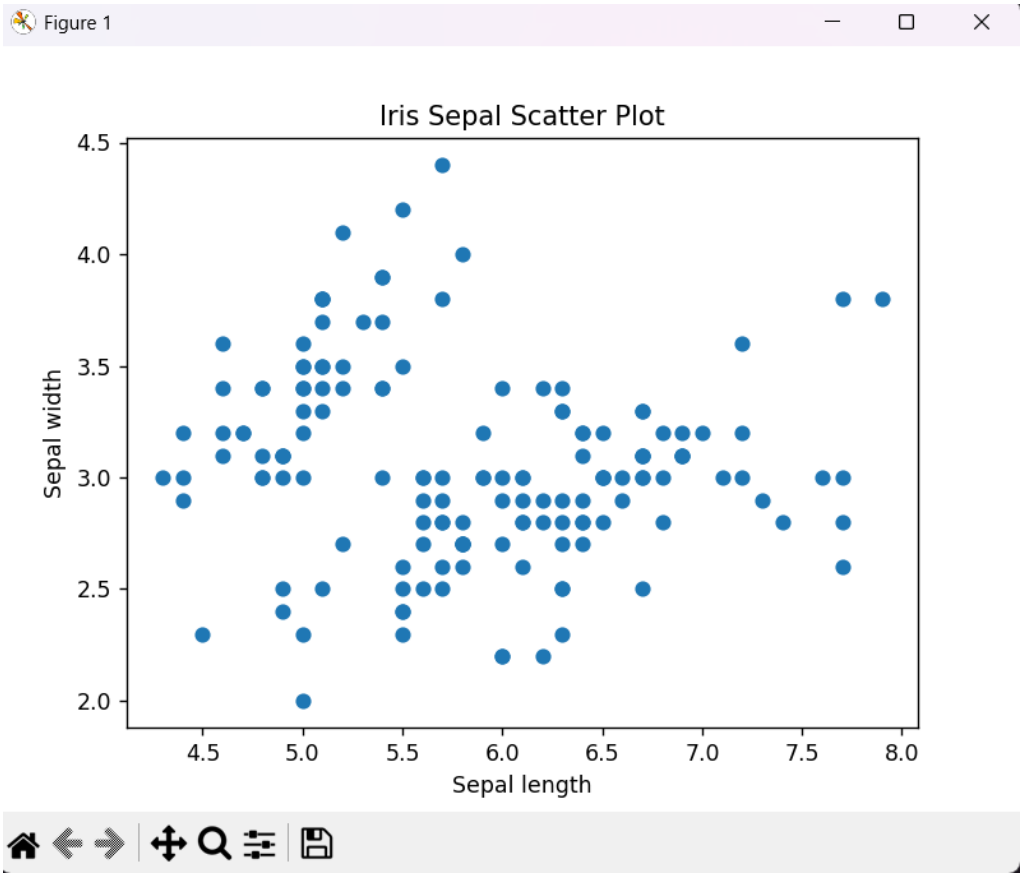
pyplot.scatter(df['sepal_length'], df['sepal_width'], c='tab:blue')
pyplot.xlabel('Sepal length')
pyplot.ylabel('Sepal width')
pyplot.title('Iris Sepal Scatter Plot')
pyplot.show()

X = df.values[:, :2]
species = df['species']
labels = dict((name, index) for index, name in enumerate(sorted(set(species))))
y = [labels[name] for name in species]

clf = svm.SVC()
clf.fit(X, y)
p = clf.predict([[5.4, 3.2]])
print(p)

```

Output:



4. Exercise 3.4

Bab3 > Code > exercise3.4.py

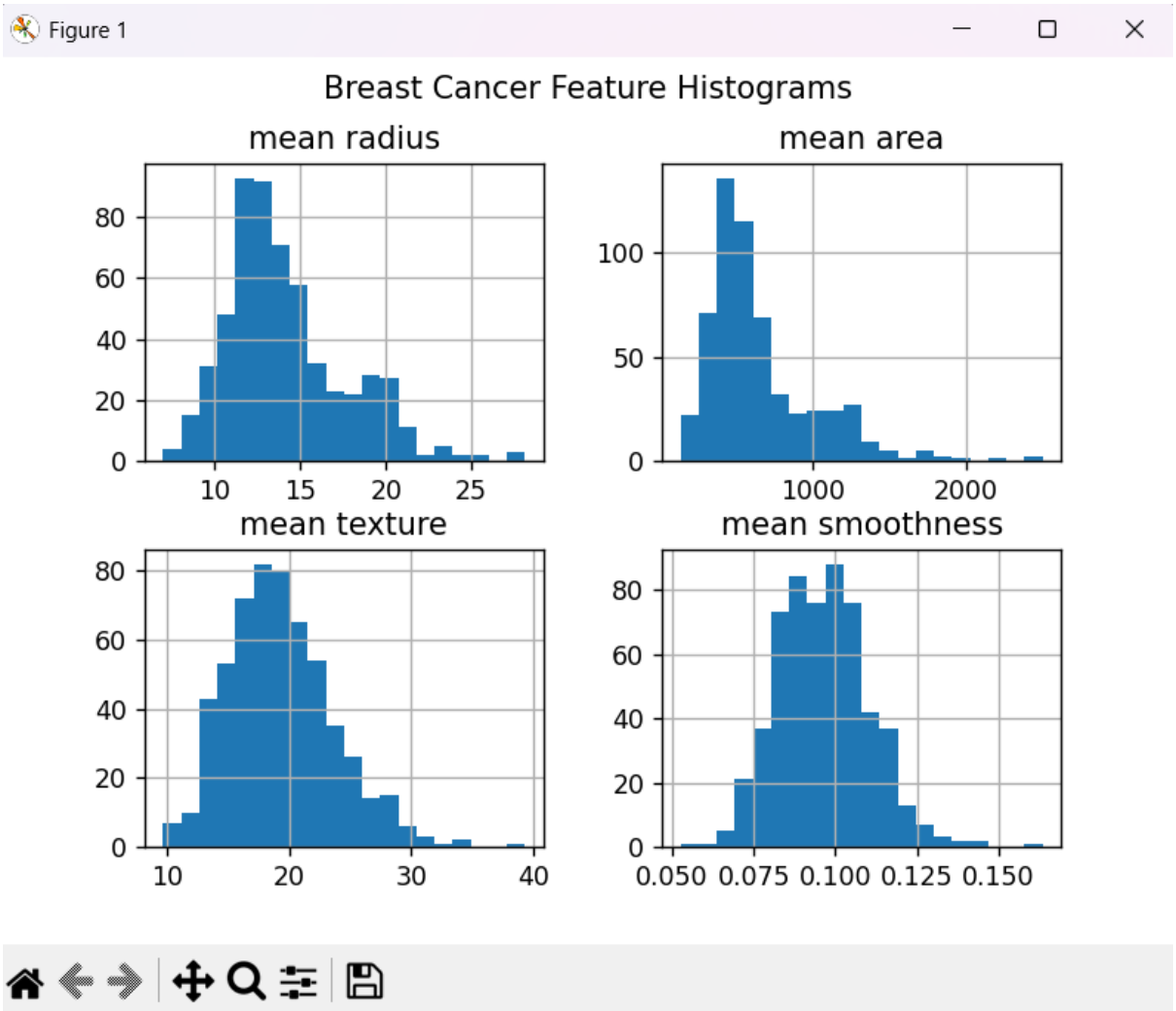
```python
# Exercise 3.4 Breast Cancer Histogram Plot
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer


cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)

selected = df[['mean radius', 'mean area', 'mean texture', 'mean smoothness']]
selected.hist(bins=20)
plt.suptitle('Breast Cancer Feature Histograms')
plt.show()

```

Output:



5. Exercise 3.5

Bab3 > Code > exercise3.5.py

```python
# Exercise 3.5 Naive Bayes Train Save Load Predi
import joblib
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB


X, y = load_iris(return_X_y=True)
clf = GaussianNB()
clf.fit(X, y)
joblib.dump(clf, 'naive_bayes_model.pkl')
clf2 = joblib.load('naive_bayes_model.pkl')
p = clf2.predict([[5.0, 3.4, 1.5, 0.4]])
print(p)

```

Output:

```
(.venv311) PS D:\AI
n Buatan/Bab3/Code/e
[0]
```

6. Exercise 3.6

Bab3 > Code > exercise3.6.py

```python
# Exercise 3.6 Linear Discriminant Analysis Classification
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis


X, y = make_classification(
    n_samples=2000,
    n_features=6,
    n_informative=3,
    n_redundant=0,
    random_state=0,
    shuffle=False,
)
print(X)
clf = LinearDiscriminantAnalysis()
clf.fit(X, y)
print(clf.predict([[0, 0, 0, 0, 0, 0]]))

```

Output:

```
(.venv311) PS D:\AI\Kecerdasan Buatan> & C:\Users\welcome\AppData\Local\Program
n_Buatan/Bab3/Code/exercise3.6.py"
 ...
 [ 0.55858636  1.20745514  1.55956083 -0.50435786 -0.32014316 -0.17555345]
 [ 0.00642618  0.93435542  2.19154679  1.1466348  -1.92120109 -0.2301177 ]
 [-0.2778356   0.81649127  2.22615366  1.11839564  0.03791157 -1.08083163]]
[0]
```

7. Exercise 3.7

Bab3 > Code > exercise3.7.py

```python
# Exercise 3.7 PCA on Breast Cancer Data
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA


cancer = load_breast_cancer()
X = cancer.data
y = cancer.target

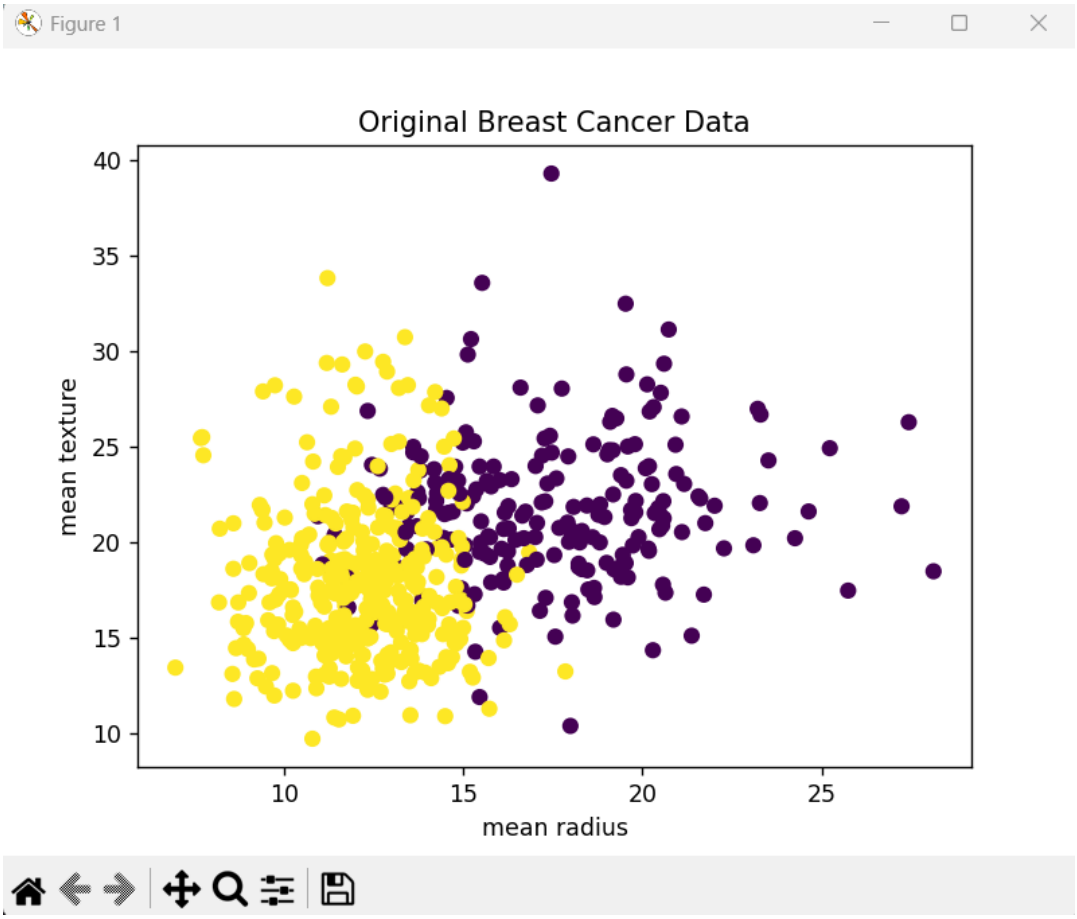
plt.figure(1)
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel('mean radius')
plt.ylabel('mean texture')
plt.title('Original Breast Cancer Data')
plt.show()

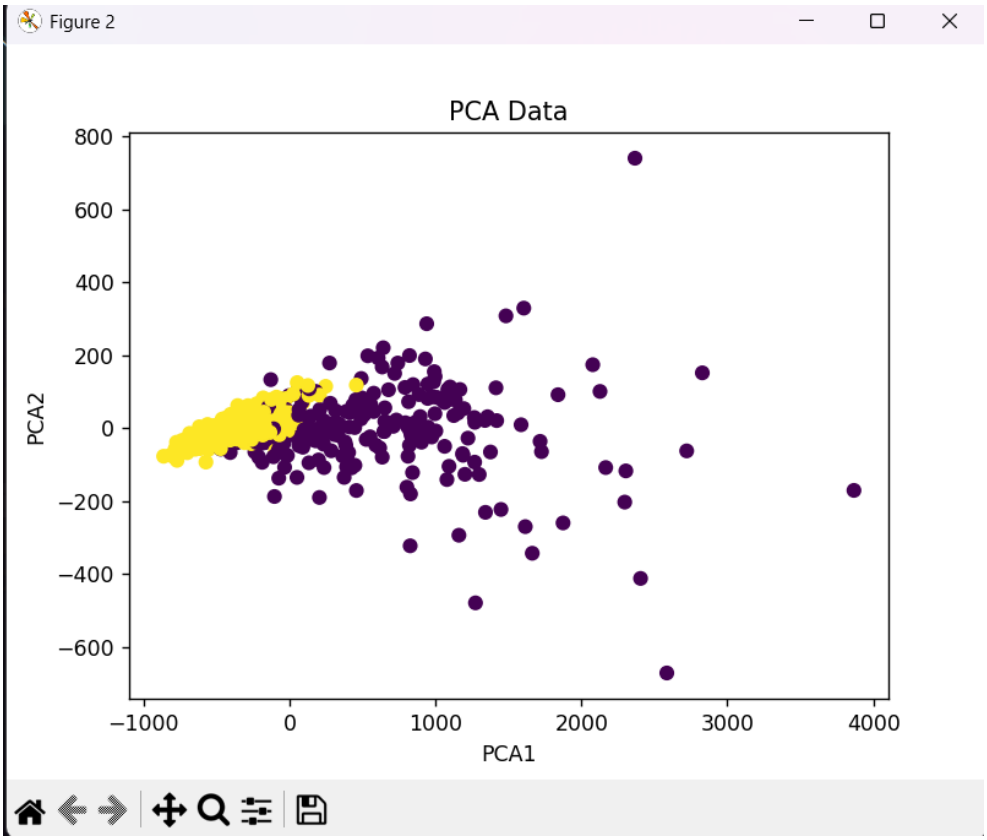
pca = PCA(n_components=2)
X1 = pca.fit_transform(X)

plt.figure(2)
plt.scatter(X1[:, 0], X1[:, 1], c=y)
plt.xlabel('PCA1')
plt.ylabel('PCA2')
plt.title('PCA Data')
plt.show()

```

Output:


Figure 1
Original Breast Cancer Data
mean texture
mean radius
40
35
30
25
20
15
10
10
15
20
25


Figure 2
PCA Data
PCA2
PCA1
800
600
400
200
0
−200
−400
−600
−1000
0
1000
2000
3000
4000

8. Exercise 3.8

Bab3 > Code > exercise3.8.py

```python
# Exercise 3.8 Decision Tree on Wine Data
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier


X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=0)
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
N = y_test.shape[0]
C = (y_test == y_pred).sum()
print("Total points: %d Correctly labeled points : %d" % (N, C))

```

Output:

```
(.venv311) PS D:\AI\Kecerdasan Buatan> & C:\Use
n Buatan/Bab3/Code/exercise3.8.py"
Total points: 89 Correctly labeled points : 77
(.venv311) PS D:\AI\Kecerdasan Buatan>
```

9. Exercise 3.9

Bab3 > Code > exercise3.9.py

```python
# Exercise 3.9 Random Forest on Diabetes Data
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split


X, y = load_diabetes(return_X_y=True)
y_class = (y > np.median(y)).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y_class, test_size=0.5, random_state=0)
clf = RandomForestClassifier(random_state=0)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Total points: %d Correctly labeled points : %d" % (y_test.shape[0], (y_test == y_pred).sum()))

```

Output:

```
(.venv311) PS D:\AI\Kecerdasan Buatan> & C:\Users
n Buatan/Bab3/Code/exercise3.9.py"
Total points: 221 Correctly labeled points : 167
```

10. Exercise 3.10

Bab3 > Code > exercise3.10.py

```python
# Exercise 3.10 Classification Comparison on Diabetes Data
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier


names = ["SVM", "Naive Bayes", "LDA", "QDA", "Decision Tree", "Random Forest", "Nearest Neighbors", "Neura
classifiers = [
    SVC(),
    GaussianNB(),
    LinearDiscriminantAnalysis(),
    QuadraticDiscriminantAnalysis(),
    DecisionTreeClassifier(),
    RandomForestClassifier(),
    KNeighborsClassifier(),
    MLPClassifier(alpha=1, max_iter=1000),
]

X, y = load_diabetes(return_X_y=True)
y_class = (y > np.median(y)).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y_class, test_size=0.5, random_state=0)

for name, clf in zip(names, classifiers):
    clf.fit(X_train, y_train)
    score = clf.score(X_test, y_test)
    print(name + ": " + str(score))

```

Outpur:

```
(.venv311) PS D:\AI\Kecerdasan Buatan> & C:\
n Buatan/Bab3/Code/exercise3.10.py"
SVM: 0.746606334841629
Naive Bayes: 0.7420814479638009
LDA: 0.746606334841629
QDA: 0.7194570135746606
Decision Tree: 0.5972850678733032
Random Forest: 0.746606334841629
Nearest Neighbors: 0.7013574660633484
Neural Networks: 0.7285067873303167
(.venv311) PS D:\AI\Kecerdasan Buatan>
```

11. Exercise 3.11

Bab3 > Code > exercise3.11.py

```python
# Exercise 3.11 Linear Regression with More Points and Plo
import matplotlib.pyplot as plt
from scipy import stats


x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
y = [3, 5, 5, 6, 7, 7, 8, 9, 10, 12]

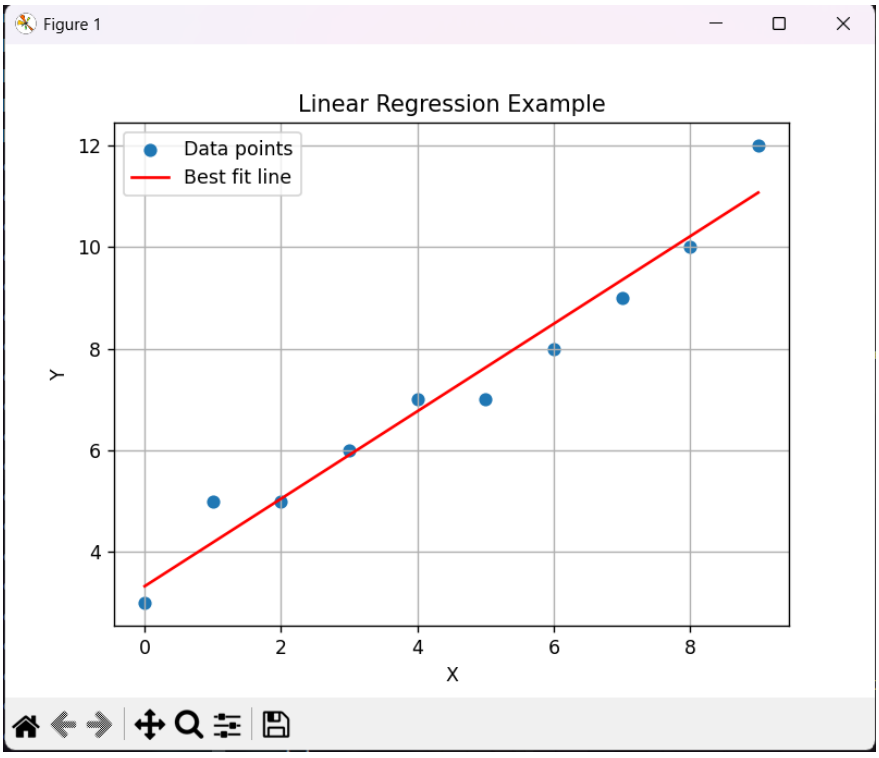
slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(value):
    return slope * value + intercept

mymodel = list(map(myfunc, x))
plt.scatter(x, y, label='Data points')
plt.plot(x, mymodel, color='red', label='Best fit line')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Linear Regression Example')
plt.legend()
plt.grid(True)
plt.show()

```

Output:



12. Excersi 3.12

Bab3 > Code > exercise3.12.py

```python
# Exercise 3.12 Multiple Linear Regression on Lin
from sklearn.datasets import load_linnerud
from sklearn import linear_model
import numpy as np


linnerud = load_linnerud()
X = linnerud.data
y = linnerud.target[:, 0]
reg = linear_model.LinearRegression()
reg.fit(X, y)
print('Coefficients: \n', reg.coef_)
print('Intercept: \n', reg.intercept_)
pred = reg.predict([X[0]])
print('Predition: \n', pred)

```

Output:

```
(.venv311) PS D:\AI\Kecerdasan Buatan> &
n Buatan/Bab3/Code/exercise3.12.py"
Coefficients:
 [-0.47502636 -0.21771647  0.09308837]
Intercept:
 208.2335188069604
Predition:
 [176.17362115]
(.venv311) PS D:\AI\Kecerdasan Buatan>
```

13. Exercise 3.13

Bab3 > Code > exercise3.13.py

```python
# Exercise 3.13 K-means with make_blobs
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs


X, y = make_blobs(n_samples=300, centers=2, cluster_std=1.0, random_state=0)
kmeans = KMeans(n_clusters=2, random_state=0).fit(X)
print(kmeans.labels_)
print(kmeans.cluster_centers_)
print(kmeans.predict([[0, 0]]))

```

Output:

```
 1 0 1 0 0 1 1 1 0 1 1 0 1 0 1 0 1 1 1 0 1 0 1 0 0 0 1 1 0 0 0 1 0 0 1 0 1
 0 1 0 0 1 0 0 1 0 0 1 1 1 0 0 0 1 1 1 1 1 1 0 1 0 0 1 1 1 0 0 1 0 0 1 0 1
 1 0 1 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 1 0 0 1 1 1 0 0 0 1 1 0 1 1
 0 1 1 0]
[[0.89921389 4.42328165]
 [1.91296081 0.76483797]]
[1]
```

14. Exercise 3.14

Bab3 > Code > exercise3.14.py

```python
# Exercise 3.14 Add Two More Points to Each Group for Label Spr
import numpy as np
from sklearn.semi_supervised import LabelSpreading


X = np.array([
    [0, 1], [1, 1], [2, 0], [3, 1], [4, 1], [5, 0],
    [10, 5], [11, 6], [12, 4], [13, 5], [14, 6], [15, 4],
])
labels = np.full(12, -1.0)
labels[0] = 0
labels[-1] = 1
print(labels)

label_spread = LabelSpreading(kernel='knn', alpha=0.8)
label_spread.fit(X, labels)
print(label_spread.transduction_)

```

Output:

```
[ 0. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.  1.]
[0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 1. 1.]
```

15. Exercise 3.15

Bab3 > Code > exercise3.15.py

```python
# Exercise 3.15 Add a Third Group of Points for Label Spreading
import numpy as np
from sklearn.semi_supervised import LabelSpreading


X = np.array([
    [0, 1], [1, 1], [2, 0], [3, 1],
    [10, 5], [11, 6], [12, 4], [13, 5],
    [20, 0], [21, 1], [22, -1], [23, 0],
])
labels = np.full(12, -1.0)
labels[0] = 0
labels[4] = 1
labels[8] = 2
print(labels)

label_spread = LabelSpreading(kernel='knn', alpha=0.8)
label_spread.fit(X, labels)
print(label_spread.transduction_)

```

Output:

```
(.venv311) PS D:\AI\Kecerdasan Buatan> & C:\Users\we
n Buatan/Bab3/Code/exercise3.15.py"
[ 0. -1. -1. -1.  1. -1. -1. -1.  2. -1. -1. -1.]
[0. 1. 1. 1. 1. 1. 2. 2. 2. 2. 2. 2.]
(.venv311) PS D:\AI\Kecerdasan Buatan>
```

16. Exercise 3.16

Bab3 > Code > exercise3.16.py

```python
# Exercise 3.16 Eight-State Routing Problem for Q-Learning
import numpy as np
import pylab as plt
from Q_Utils import *


points_list = [(0, 1), (1, 2), (1, 3), (2, 4), (3, 5), (4, 6), (5, 7), (3, 6)]
goal = 7
showgraph(points_list)
MATRIX_SIZE = 8
R = createRmat(MATRIX_SIZE, points_list, goal)
Q = np.matrix(np.zeros([MATRIX_SIZE, MATRIX_SIZE]))
gamma = 0.8

scores = []
for _ in range(700):
    current_state = np.random.randint(0, int(Q.shape[0]))
    available_act = available_actions(R, current_state)
    action = sample_next_action(available_act)
    score = update(R, Q, current_state, action, gamma)
    scores.append(score)

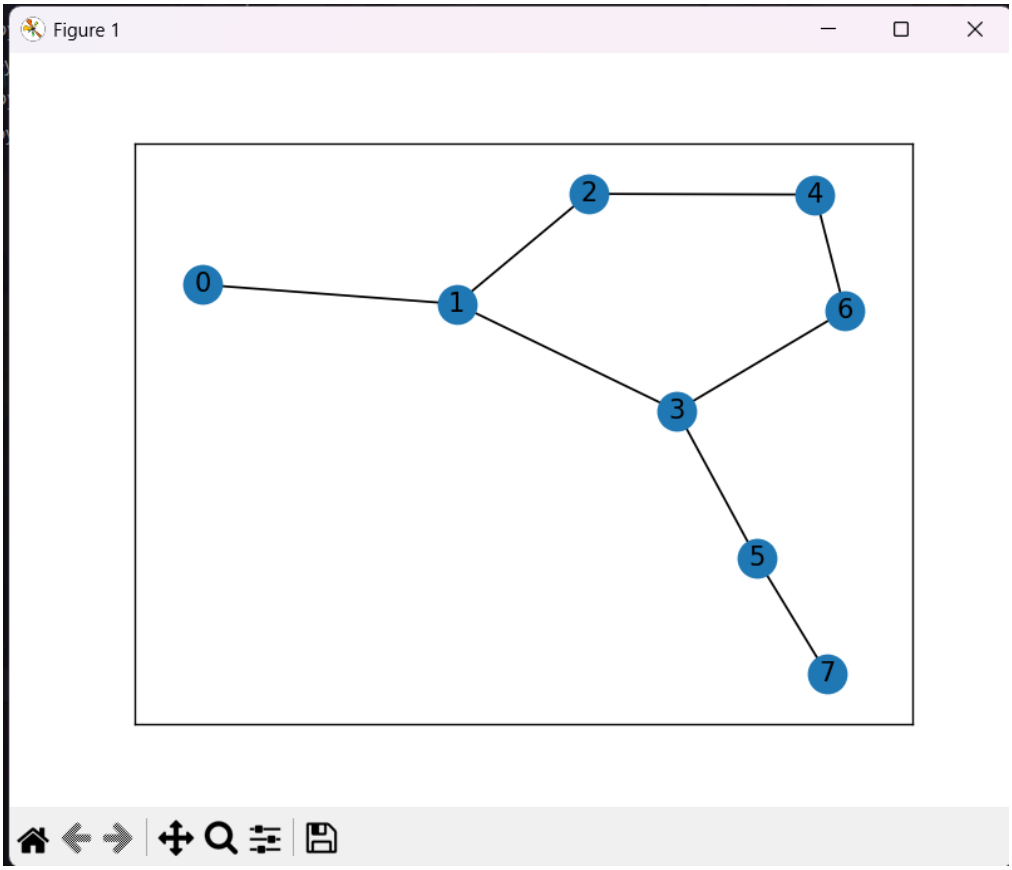
print('Trained Q matrix:')
print(Q / np.max(Q) * 100)

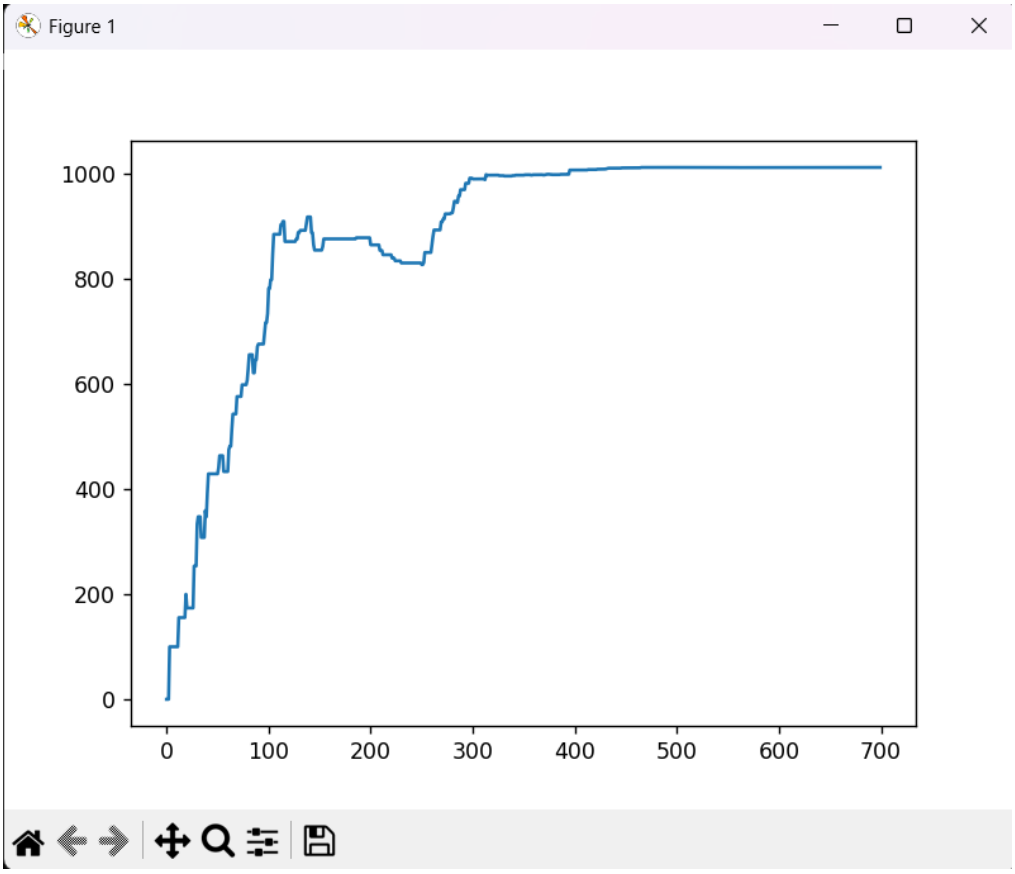
current_state = 0
steps = [current_state]
while current_state != goal:
    next_step_index = np.where(Q[current_state,] == np.max(Q[current_state,]))[1]
    if next_step_index.shape[0] > 1:
        next_step_index = int(np.random.choice(next_step_index, size=1))
    else:
        next_step_index = int(next_step_index)
    steps.append(next_step_index)
    current_state = next_step_index

print('Most efficient path:')
print(steps)
plt.plot(scores)
plt.show()

```

Output:


Figure 1
0
1
2
3
4
5
6
7


Figure 1
1000
800
600
400
200
0
0
100
200
300
400
500
600
700

17. Exercise 3.17

Bab3 > Code > exercise3.17.py

```python
# Exercise 3.17 Add KNN to Ensemble Learning
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC


iris = datasets.load_iris()
X, y = iris.data[:, 1:3], iris.target
clf1 = LogisticRegression(random_state=1)
clf2 = RandomForestClassifier(n_estimators=50, random_state=1)
clf3 = GaussianNB()
clf4 = SVC()
clf5 = KNeighborsClassifier()
eclf = VotingClassifier(
    estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3), ('svc', clf4), ('knn', clf5)],
    voting='hard',
)

for clf, label in zip(
    [clf1, clf2, clf3, clf4, clf5, eclf],
    ['Logistic Regression', 'Random Forest', 'naive Bayes', 'SVM', 'KNN', 'Ensemble'],
):
    scores = cross_val_score(clf, X, y, scoring='accuracy', cv=5)
    print("Accuracy: %0.2f (+/- %0.2f) [%s]" % (scores.mean(), scores.std(), label))

```

Output:

```
n Buatan/Bab3/Code/exercise3.17.py"
Accuracy: 0.95 (+/- 0.04) [Logistic Regression]
Accuracy: 0.94 (+/- 0.04) [Random Forest]
Accuracy: 0.91 (+/- 0.04) [naive Bayes]
Accuracy: 0.95 (+/- 0.04) [SVM]
Accuracy: 0.95 (+/- 0.04) [KNN]
Accuracy: 0.95 (+/- 0.04) [Ensemble]
(.venv311) PS D:\AI\Kecerdasan Buatan>
```

18. Exercise 3.18

Bab3 > Code > exercise3.18.py

```python
# Exercise 3.18 Regression on California Housing Dataset
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split

housing = fetch_california_housing(as_frame=True)
df = housing.frame.copy().rename(columns={'MedHouseVal': 'target'})

X = df.drop(columns=['target'])
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(f'R2 score: {r2_score(y_test, pred):.4f}')

```

Output:

```
(.venv311) PS D:\AI\Kecerdasan Bua
n Buatan/Bab3/Code/exercise3.18.py
R2 score: 0.8062
```

19. Exercise 3.19

Bab3 > Code > exercise3.19.py

```python
# Exercise 3.19 PyCaret Breast Cancer Classification
import pandas as pd
from sklearn.datasets import load_breast_cancer


cancer = load_breast_cancer(as_frame=True)
df = cancer.frame.copy()
df['Target'] = cancer.target
from pycaret import classification

classification.setup(data=df, target='Target')
classification.compare_models()

```

Output:

```
(.venv311) PS D:\AI\Kecerdasan Buatan> &
n Buatan/Bab3/Code/exercise3.19.py"
et      1.0000  1.0000  1.0000     0.088
rf      0.9980  0.9947  0.9949     0.113
lr      0.9940  0.9839  0.9841     0.965
lda     0.9658  0.8993  0.9050     0.017
knn     0.9518  0.8627  0.8687     0.033
svm     0.8670  0.6938  0.7195     0.016
dummy   0.7716  0.0000  0.0000     0.026
```

20. Exercise 3.20

Bab3 > Code > exercise3.20.py

```python
# Exercise 3.20 LazyPredict Classification on a Different Dataset
from lazypredict.Supervised import LazyClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split


X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.25, random_state=1)
clf = LazyClassifier()
models, predictions = clf.fit(X_train, X_test, y_train, y_test)
print(models)

```

Output:

```
RandomForestClassifier          0.944056           0.937500  0.979545  0.943852  0.944041  0.944056  0.199228
LinearDiscriminantAnalysis      0.951049           0.936364  0.979132  0.950322  0.954656  0.951049  0.018487
BaggingClassifier               0.930070           0.919318  0.968182  0.929527  0.930553  0.930070  0.060081
NearestCentroid                 0.930070           0.915909  0.915909  0.929205  0.931928  0.930070  0.013486
BernoulliNB                     0.909091           0.902273  0.965909  0.908931  0.908864  0.909091  0.014613
ExtraTreeClassifier             0.909091           0.895455  0.895455  0.908182  0.909677  0.909091  0.011193
DummyClassifier                 0.615385           0.500000  0.500000  0.468864  0.378698  0.615385  0.017750
(.venv311) PS D:\AI\Kecerdasan Buatan>
```

21. Exercise 3.21

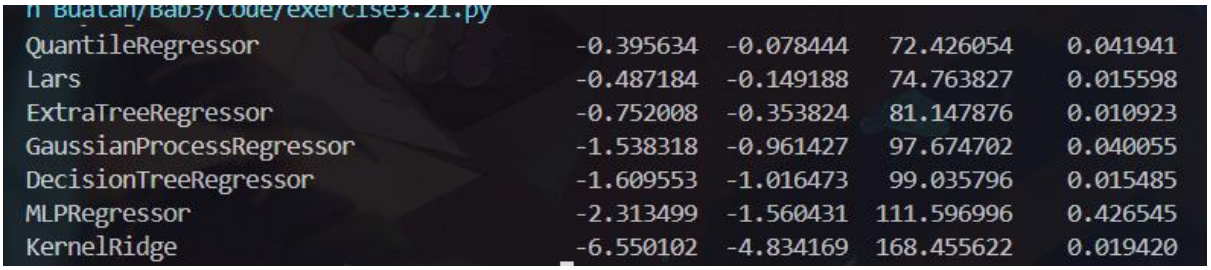
Bab3 > Code > exercise3.21.py

```python
# Exercise 3.21 LazyPredict Regression on a Different Dataset
from lazypredict.Supervised import LazyRegressor
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split


X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.1, random_state=1)
reg = LazyRegressor()
models, predictions = reg.fit(X_train, X_test, y_train, y_test)
print(models)

```

Output:


n Buatan/Bab3/Code/exercise3.21.py
QuantileRegressor -0.395634 -0.078444 72.426054 0.041941
Lars -0.487184 -0.149188 74.763827 0.015598
ExtraTreeRegressor -0.752008 -0.353824 81.147876 0.010923
GaussianProcessRegressor -1.538318 -0.961427 97.674702 0.040055
DecisionTreeRegressor -1.609553 -1.016473 99.035796 0.015485
MLPRegressor -2.313499 -1.560431 111.596996 0.426545
KernelRidge -6.550102 -4.834169 168.455622 0.019420